

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО
ITMO University

ЛАБОРАТОРНАЯ РАБОТА №3

По дисциплине: Алгоритмы и структуры данных

Тема работы: Алгоритмы сортировки.

Обучающиеся: Казанцев Александр Сергеевич

Факультет: Факультет прикладной информатики

Направление подготовки: 11.04.02 Инфокоммуникационные технологии и системы связи

Группа: К4112с

Руководитель: Мусаев Андрей Александрович

Санкт-Петербург, 2026

СОДЕРЖАНИЕ

	Стр.
ВВЕДЕНИЕ	3
1 ОСНОВНОЕ ЗАДАНИЕ	4
1.1 Задание 1	4
1.2 Задание 2	9
1.3 Задание 3	14
ЗАКЛЮЧЕНИЕ	15
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	15

ВВЕДЕНИЕ

В этой лабораторке я хочу на практике разобраться, как работают разные алгоритмы сортировки и какая у них реальная вычислительная сложность. В бэкенде это суперважная тема: если засунуть в проект неоптимальную сортировку, то при нормальной нагрузке сервер просто ляжет. Чтобы разложить всё по полочкам, я напишу пять отдельных программ на Go, посмотрю, как они справляются с нашими базами данных студентов ИТМО при росте объёмов, и в конце наглядно сравню, какие методы реально эффективные, а какие лучше вообще никогда не трогать.

1 ОСНОВНОЕ ЗАДАНИЕ

1.1 Задание 1

Условие:

Написать программу с функциями для быстрой сортировки, сортировки слиянием и сортировки расческой.

Общая логика решения:

В ИТМО до сих пор куча народу ждет места в общагах, поэтому я решил продолжить развивать бэкенд из прошлых лабораторных. В этот раз я учу систему самостоятельно упорядочивать списки студентов, чтобы по ним потом можно было быстро искать данные. Базу студентов я взял ту же самую, со всеми реальными именами. Цель программ в этом задании это взять этот хаотичный список и красиво раскидать его по порядку, от меньшего номера очереди к большему.

1. Быстрая сортировка (Quick Sort)

Этот алгоритм работает по принципу **разделяй и властвуй**. Чтобы бэкенд не зависал на больших объемах данных, если список вдруг пришел уже частично отсортированным, я сделал выбор опорного элемента строго по центру массива. Программа раскидывает студентов по трем отдельным корзинам (кто меньше опорного, кто равен ему и кто больше), а потом рекурсивно склеивает всё обратно.

Реализация Quick Sort на языке Go:

```
main.go .../01_quick_sort X main.go .../02_merge_sort X main.go .../03_comb_sort X
lab-03-sorting > 01_quick_sort > main.go
1 package main
2
3 import "fmt"
4
5 type Student struct {
6     Name      string
7     IsMagistr bool
8     InQueue   bool
9     QueueNumber int
10 }
11
12 // quickSortStudents - быстрая сортировка студентов по номеру в очереди
13 func quickSortStudents(students []Student) []Student {
14     if len(students) < 2 {
15         return students
16     }
17
18     midIndex := len(students) / 2
19     pivot := students[midIndex]
20
21     var less []Student
22     var equal []Student
23     var greater []Student
24
25     for _, s := range students {
26         if s.QueueNumber < pivot.QueueNumber {
27             less = append(less, s)
28         } else if s.QueueNumber > pivot.QueueNumber {
29             greater = append(greater, s)
30         } else {
31             equal = append(equal, s)
32         }
33     }
34
35     result := append(quickSortStudents(less), equal...)
36     result = append(result, quickSortStudents(greater)...)
37
38     return result
39 }
40
```

```

41 func main() {
42     groupData := []Student{
43         {Name: "Егорова Мария", IsMagistr: true, InQueue: true, QueueNumber:
44             150},
45         {Name: "Фролова Кристина", IsMagistr: true, InQueue: false,
46             QueueNumber: 0},
47         {Name: "Костюченко Тимофей", IsMagistr: false, InQueue: true,
48             QueueNumber: 342},
49         {Name: "Портнова Ксения", IsMagistr: true, InQueue: true, QueueNumber:
50             612},
51         {Name: "Кепжаев Рахим", IsMagistr: true, InQueue: true, QueueNumber:
52             89},
53         {Name: "Даньшин Семён", IsMagistr: true, InQueue: true, QueueNumber:
54             540},
55         {Name: "Шихайло Сергей", IsMagistr: false, InQueue: false, QueueNumber:
56             0},
57         {Name: "Бабушкин Александр", IsMagistr: true, InQueue: true,
58             QueueNumber: 221},
59         {Name: "Якунин Андрей", IsMagistr: true, InQueue: true, QueueNumber:
60             431},
61         {Name: "Казанцев Александр", IsMagistr: true, InQueue: true,
62             QueueNumber: 641},
63         {Name: "Юркин Александр", IsMagistr: true, InQueue: true, QueueNumber:
64             112},
65         {Name: "Панкратова Анна", IsMagistr: false, InQueue: true, QueueNumber:
66             299},
67         {Name: "Колесникова Лариса", IsMagistr: true, InQueue: false,
68             QueueNumber: 0},
69     }
70
71     fmt.Println("Исходный список:")
72     for _, s := range groupData {
73         fmt.Printf("Студент: %-20s | Очередь: %d\n", s.Name, s.QueueNumber)
74     }
75     fmt.Println()
76
77     sortedData := quickSortStudents(groupData)
78     fmt.Println("Отсортированный список:")
79     for _, s := range sortedData {
80         fmt.Printf("Студент: %-20s | Очередь: %d\n", s.Name, s.QueueNumber)
81     }
82 }

```

```

→ 01_quick_sort go run main.go
Исходный список:
Студент: Егорова Мария | Очередь: 150
Студент: Фролова Кристина | Очередь: 0
Студент: Костюченко Тимофей | Очередь: 342
Студент: Портнова Ксения | Очередь: 612
Студент: Кепжаев Рахим | Очередь: 89
Студент: Даньшин Семён | Очередь: 540
Студент: Шихайло Сергей | Очередь: 0
Студент: Бабушкин Александр | Очередь: 221
Студент: Якунин Андрей | Очередь: 431
Студент: Казанцев Александр | Очередь: 641
Студент: Юркин Александр | Очередь: 112
Студент: Панкратова Анна | Очередь: 299
Студент: Колесникова Лариса | Очередь: 0

Отсортированный список:
Студент: Фролова Кристина | Очередь: 0
Студент: Шихайло Сергей | Очередь: 0
Студент: Колесникова Лариса | Очередь: 0
Студент: Кепжаев Рахим | Очередь: 89
Студент: Юркин Александр | Очередь: 112
Студент: Егорова Мария | Очередь: 150
Студент: Бабушкин Александр | Очередь: 221
Студент: Панкратова Анна | Очередь: 299
Студент: Костюченко Тимофей | Очередь: 342
Студент: Якунин Андрей | Очередь: 431
Студент: Даньшин Семён | Очередь: 540

```

2. Сортировка слиянием (Merge Sort)

Смысл этой сортировки для меня в том, что она гарантирует стабильную скорость при любых условиях под нагрузкой бэкенда. Мой код рекурсивно пилит массив студентов ровно пополам, пока не дойдет до одиночных элементов. После этого включается функция слияния, которая аккуратно обходит эти половинки и склеивает их в один общий правильный лог. Алгоритм работает четко, но требует выделения дополнительной памяти под новые слайсы.

Реализация Merge Sort на языке Go:

```
1 package main
2
3 import "fmt"
4
5 type Student struct {
6     Name      string
7     IsMagistr  bool
8     InQueue   bool
9     QueueNumber int
10 }
11
12 // merge объединяет два предварительно отсортированных слайса (left и right) в
13 // один общий упорядоченный
14 func merge(left, right []Student) []Student {
15     result := make([]Student, 0, len(left)+len(right))
16     i, j := 0, 0
17
18     for i < len(left) && j < len(right) {
19         if left[i].QueueNumber <= right[j].QueueNumber {
20             result = append(result, left[i])
21             i++
22         } else {
23             result = append(result, right[j])
24             j++
25         }
26     }
27
28     for i < len(left) {
29         result = append(result, left[i])
30         i++
31     }
32
33     for j < len(right) {
34         result = append(result, right[j])
35         j++
36     }
37
38     return result
39 }
40
41 // mergeSortStudents рекурсивно разбиваем массив пополам, пока длина
42 // подмассивов не станет меньше 2
43 func mergeSortStudents(students []Student) []Student {
44     if len(students) < 2 {
45         return students
46     }
47
48     mid := len(students) / 2
49
50     left := mergeSortStudents(students[:mid])
51     right := mergeSortStudents(students[mid:])
52
53     return merge(left, right)
54 }
```

```

54 func main() {
55     groupData := []Student{
56         {Name: "Егорова Мария", IsMagistr: true, InQueue: true, QueueNumber:
57             150},
58         {Name: "Фролова Кристина", IsMagistr: true, InQueue: false,
59             QueueNumber: 0},
60         {Name: "Костюченко Тимофей", IsMagistr: false, InQueue: true,
61             QueueNumber: 342},
62         {Name: "Портнова Ксения", IsMagistr: true, InQueue: true, QueueNumber:
63             612},
64         {Name: "Кепжаев Рахим", IsMagistr: true, InQueue: true, QueueNumber:
65             89},
66         {Name: "Даньшин Семён", IsMagistr: true, InQueue: true, QueueNumber:
67             540},
68         {Name: "Шихайло Сергей", IsMagistr: false, InQueue: false, QueueNumber:
69             0},
70         {Name: "Бабушкин Александр", IsMagistr: true, InQueue: true,
71             QueueNumber: 221},
72         {Name: "Якунин Андрей", IsMagistr: true, InQueue: true, QueueNumber:
73             431},
74         {Name: "Казанцев Александр", IsMagistr: true, InQueue: true,
75             QueueNumber: 641},
76         {Name: "Юркин Александр", IsMagistr: true, InQueue: true, QueueNumber:
77             112},
78         {Name: "Панкратова Анна", IsMagistr: false, InQueue: true, QueueNumber:
79             299},
80         {Name: "Колесникова Лариса", IsMagistr: true, InQueue: false,
81             QueueNumber: 0},
82     }
83 }

70
71 fmt.Println("Исходный список:")
72 for _, s := range groupData {
73     fmt.Printf("Студент: %-20s | Очередь: %d\n", s.Name, s.QueueNumber)
74 }
75 fmt.Println()
76
77 sortedData := mergeSortStudents(groupData)
78
79 fmt.Println("Отсортированный список:")
80 for _, s := range sortedData {
81     fmt.Printf("Студент: %-20s | Очередь: %d\n", s.Name, s.QueueNumber)
82 }
83 }

```

```

-> 02_merge_sort go run main.go
Исходный список:
Студент: Егорова Мария | Очередь: 150
Студент: Фролова Кристина | Очередь: 0
Студент: Костюченко Тимофей | Очередь: 342
Студент: Портнова Ксения | Очередь: 612
Студент: Кепжаев Рахим | Очередь: 89
Студент: Даньшин Семён | Очередь: 540
Студент: Шихайло Сергей | Очередь: 0
Студент: Бабушкин Александр | Очередь: 221
Студент: Якунин Андрей | Очередь: 431
Студент: Казанцев Александр | Очередь: 641
Студент: Юркин Александр | Очередь: 112
Студент: Панкратова Анна | Очередь: 299
Студент: Колесникова Лариса | Очередь: 0

Отсортированный список:
Студент: Фролова Кристина | Очередь: 0
Студент: Шихайло Сергей | Очередь: 0
Студент: Колесникова Лариса | Очередь: 0
Студент: Кепжаев Рахим | Очередь: 89
Студент: Юркин Александр | Очередь: 112
Студент: Егорова Мария | Очередь: 150
Студент: Бабушкин Александр | Очередь: 221
Студент: Панкратова Анна | Очередь: 299
Студент: Костюченко Тимофей | Очередь: 342
Студент: Якунин Андрей | Очередь: 431
Студент: Даньшин Семён | Очередь: 540

```

3. Сортировка расчёской (Comb Sort)

Я реализовал этот алгоритм как продвинутый вариант обычной пузырьковой сортировки. Вместо того чтобы сравнивать только соседние элементы, моя **расчёска** начинает с больших шагов, которые на каждом этапе уменьшаются на коэффициент **1.3**. Это позволяет быстро сдвинуть мелкие номера очередей из самого конца списка в начало. Большой плюс в том, что массив перестраивается прямо на месте без создания лишних слайсов.

Реализация Comb Sort на языке Go:

```
1 package main
2
3 import "fmt"
4
5 type Student struct {
6     Name      string
7     IsMagistr  bool
8     InQueue   bool
9     QueueNumber int
10 }
11
12 // combSortStudents - сортировка расчёской массива студентов по номеру в очереди
13 func combSortStudents(students []Student) {
14     n := len(students)
15     gap := n
16     // оптимальный коэффициент уменьшения шага
17     shrink := 1.3
18     // флаг - были ли перестановки на текущей итерации
19     swapped := true
20
21     for gap > 1 || swapped {
22         gap = int(float64(gap) / shrink)
23         if gap < 1 {
24             gap = 1
25         }
26
27         swapped = false
28
29         for i := 0; i < n-gap; i++ {
30             if students[i].QueueNumber > students[i+gap].QueueNumber {
31                 students[i], students[i+gap] = students[i+gap], students[i]
32                 swapped = true
33             }
34         }
35     }
36 }
37
38 func main() {
39     groupData := []Student{
40         {Name: "Егорова Мария", IsMagistr: true, InQueue: true, QueueNumber:
41             150},
42         {Name: "Фролова Кристина", IsMagistr: true, InQueue: false,
43             QueueNumber: 0},
44         {Name: "Костюченко Тимофей", IsMagistr: false, InQueue: true,
45             QueueNumber: 342},
46         {Name: "Портнова Ксения", IsMagistr: true, InQueue: true, QueueNumber:
47             612},
48         {Name: "Кепжаев Рахим", IsMagistr: true, InQueue: true, QueueNumber:
49             89},
50         {Name: "Даньшин Семён", IsMagistr: true, InQueue: true, QueueNumber:
51             540},
52         {Name: "Шихайло Сергей", IsMagistr: false, InQueue: false, QueueNumber:
53             0},
54         {Name: "Бабушкин Александр", IsMagistr: true, InQueue: true,
55             QueueNumber: 221},
56         {Name: "Якунин Андрей", IsMagistr: true, InQueue: true, QueueNumber:
57             431},
58         {Name: "Казанцев Александр", IsMagistr: true, InQueue: true,
59             QueueNumber: 641},
60         {Name: "Юркин Александр", IsMagistr: true, InQueue: true, QueueNumber:
61             112},
62         {Name: "Панкратова Анна", IsMagistr: false, InQueue: true, QueueNumber:
63             299},
64         {Name: "Колесникова Лариса", IsMagistr: true, InQueue: false,
65             QueueNumber: 0},
66     }
```



```

54     fmt.Println("Исходный список:")
55     for _, s := range groupData {
56         fmt.Printf("Студент: %-20s | Очередь: %d\n", s.Name, s.QueueNumber)
57     }
58     fmt.Println()
59     combSortStudents(groupData)
60
61     fmt.Println("Отсортированный список:")
62     for _, s := range groupData {
63         fmt.Printf("Студент: %-20s | Очередь: %d\n", s.Name, s.QueueNumber)
64     }
65 }
66
67 }

```

```

➔ 03_comb_sort go run main.go

```

```

Исходный список:
Студент: Егорова Мария | Очередь: 150
Студент: Фролова Кристина | Очередь: 0
Студент: Костюченко Тимофей | Очередь: 342
Студент: Портнова Ксения | Очередь: 612
Студент: Кепжаев Рахим | Очередь: 89
Студент: Даньшин Семён | Очередь: 540
Студент: Шихайло Сергей | Очередь: 0
Студент: Бабушкин Александр | Очередь: 221
Студент: Якунин Андрей | Очередь: 431
Студент: Казанцев Александр | Очередь: 641
Студент: Юркин Александр | Очередь: 112
Студент: Панкратова Анна | Очередь: 299
Студент: Колесникова Лариса | Очередь: 0

```

```

Отсортированный список:
Студент: Колесникова Лариса | Очередь: 0
Студент: Фролова Кристина | Очередь: 0
Студент: Шихайло Сергей | Очередь: 0
Студент: Кепжаев Рахим | Очередь: 89
Студент: Юркин Александр | Очередь: 112
Студент: Егорова Мария | Очередь: 150
Студент: Бабушкин Александр | Очередь: 221
Студент: Панкратова Анна | Очередь: 299
Студент: Костюченко Тимофей | Очередь: 342
Студент: Якунин Андрей | Очередь: 431
Студент: Даньшин Семён | Очередь: 540
Студент: Портнова Ксения | Очередь: 612
Студент: Казанцев Александр | Очередь: 641

```

```

➔ 03_comb_sort

```

1.2 Задание 2

Условие:

Изучить блочную и пирамидальную сортировку. Написать соответствующие программы.

Общая логика решения:

Для дальнейшего развития студенческого бэкенда я решил протестировать другие подходы к упорядочиванию данных. Базу студентов ИТМО я оставил абсолютно той же самой чтобы все было честно. Моя задача это взять эту кашу из номеров и выстроить всех строго по порядку, но теперь я буду мучить этот список кучей и блоками. Посмотрим насколько удобно и логично это получится переписать на Go.

1. Пирамидальная сортировка (Heap Sort)

Эту сортировку я реализовал как крутой движок для работы с приоритетами. Смысл в том что программа строит из массива студентов специальное двоичное дерево - **кучу**. Эту структуру я отдельно нарисовал ниже для наглядности чтобы показать как номера очередей распределяются по вершинам и узлам.. В этой куче самый большой номер очереди автоматически всплывает на самую вершину. Потом я просто забираю этот максимум в конец списка и перестраиваю структуру заново для оставшихся элементов. Алгоритм работает стабильно быстро при любой нагрузке и не требует выделения лишней памяти под новые слайсы.

Моя карточка с номером 641 вылезла на самый верх, потому что это максимальное число в списке. Алгоритм специально поднял меня на вершину, чтобы первым делом забрать этот элемент и выкинуть его в самый конец. Это логично, ведь с таким номером я стою в самом хвосте очереди на заселение. Сразу после этого дерево автоматически перестроится, и на вершину поднимется Портнова с номером 612, затем Даньшин с номером 540, и так далее по убыванию.



Реализация Heap Sort на языке Go:

```

1 package main
2
3 import "fmt"
4
5 type Student struct {
6     Name      string
7     IsMagistr bool
8     InQueue   bool
9     QueueNumber int
10 }
11
12 // heapify — перемещает элемент с маленьким приоритетом вниз, если его потомки
    больше
13 func heapify(students []Student, root, size int) {
14     for {
15         left := (root << 1) + 1
16         right := left + 1
17         largest := root // временно считаю корень самым большим элементом
18
19         if left < size && students[left].QueueNumber > students[largest].QueueNumber {
20             largest = left
21         }
22
23         if right < size && students[right].QueueNumber > students[largest].QueueNumber {
24             largest = right
25         }
26
27         if largest == root {
28             break
29         }
30
31         students[root], students[largest] = students[largest], students[root]
32         root = largest
33     }
34 }
35
36 // heapSortStudents — пирамидальная сортировка
37 func heapSortStudents(students []Student) {
38     n := len(students)
39
40     for i := (n >> 1) - 1; i >= 0; i-- {
41         heapify(students, i, n)
42     }
43
44     for i := n - 1; i > 0; i-- {
45         students[0], students[i] = students[i], students[0]
46         heapify(students, 0, i)
47     }
48 }
49
50 }
51

```

```
lab-03-sorting > 04_heap_sort > main.go
52 func main() {
53     groupData := []Student{
54         {Name: "Егорова Мария", IsMagistr: true, InQueue: true, QueueNumber:
55             150},
56         {Name: "Фролова Кристина", IsMagistr: true, InQueue: false,
57             QueueNumber: 0},
58         {Name: "Костюченко Тимофей", IsMagistr: false, InQueue: true,
59             QueueNumber: 342},
60         {Name: "Портнова Ксения", IsMagistr: true, InQueue: true, QueueNumber:
61             612},
62         {Name: "Кепжаев Рахим", IsMagistr: true, InQueue: true, QueueNumber:
63             89},
64         {Name: "Даньшин Семён", IsMagistr: true, InQueue: true, QueueNumber:
65             540},
66         {Name: "Шихайло Сергей", IsMagistr: false, InQueue: false, QueueNumber:
67             0},
68         {Name: "Бабушкин Александр", IsMagistr: true, InQueue: true,
69             QueueNumber: 221},
70         {Name: "Якунин Андрей", IsMagistr: true, InQueue: true, QueueNumber:
71             431},
72         {Name: "Казанцев Александр", IsMagistr: true, InQueue: true,
73             QueueNumber: 641},
74         {Name: "Юркин Александр", IsMagistr: true, InQueue: true, QueueNumber:
75             112},
76         {Name: "Панкратова Анна", IsMagistr: false, InQueue: true, QueueNumber:
77             299},
78         {Name: "Колесникова Лариса", IsMagistr: true, InQueue: false,
79             QueueNumber: 0},
80     }
81 }

82 fmt.Println("Исходный список:")
83 for _, s := range groupData {
84     fmt.Printf("Студент: %-20s | Очередь: %d\n", s.Name, s.QueueNumber)
85 }
86 fmt.Println()
87 heapSortStudents(groupData)
88
89 fmt.Println("Отсортированный список:")
90 for _, s := range groupData {
91     fmt.Printf("Студент: %-20s | Очередь: %d\n", s.Name, s.QueueNumber)
92 }
93 }
```

```
04_heap_sort go run main.go
Исходный список:
Студент: Егорова Мария | Очередь: 150
Студент: Фролова Кристина | Очередь: 0
Студент: Костюченко Тимофей | Очередь: 342
Студент: Портнова Ксения | Очередь: 612
Студент: Кепжаев Рахим | Очередь: 89
Студент: Даньшин Семён | Очередь: 540
Студент: Шихайло Сергей | Очередь: 0
Студент: Бабушкин Александр | Очередь: 221
Студент: Якунин Андрей | Очередь: 431
Студент: Казанцев Александр | Очередь: 641
Студент: Юркин Александр | Очередь: 112
Студент: Панкратова Анна | Очередь: 299
Студент: Колесникова Лариса | Очередь: 0

Отсортированный список:
Студент: Фролова Кристина | Очередь: 0
Студент: Шихайло Сергей | Очередь: 0
Студент: Колесникова Лариса | Очередь: 0
Студент: Кепжаев Рахим | Очередь: 89
Студент: Юркин Александр | Очередь: 112
Студент: Егорова Мария | Очередь: 150
Студент: Бабушкин Александр | Очередь: 221
Студент: Панкратова Анна | Очередь: 299
Студент: Костюченко Тимофей | Очередь: 342
Студент: Якунин Андрей | Очередь: 431
Студент: Даньшин Семён | Очередь: 540
Студент: Портнова Ксения | Очередь: 612
Студент: Казанцев Александр | Очередь: 641
04_heap_sort
```

2. Блочная сортировка (Bucket Sort)

Универсальный вариант этой сортировки вообще не дает никаких преимуществ и в реальных проектах им никогда не пользуются. Чтобы показать нормальное понимание того как устроен алгоритм я просто адаптировал его под логику нашей общаги. Я распределил студентов по трем разным корзинам в зависимости от их номеров очередей. В первую корзину у меня уходят номера до 200, во вторую от 201 до 500, а в третью все остальные тяжелые случаи. Внутри каждой корзины я быстро упорядочил ребят, а потом просто склеил эти три блока в один чистый итоговый список.

Реализация Bucket Sort на языке Go:

```
1 package main
2
3 import "fmt"
4
5 type Student struct {
6     Name      string
7     IsMagistr bool
8     InQueue   bool
9     QueueNumber int
10 }
11
12 // bucketSortStudents - блочная сортировка студентов по номерам в очереди
13 func bucketSortStudents(students []Student) []Student {
14     if len(students) == 0 {
15         return students
16     }
17
18     // корзины под диапазоны номеров очередей
19     var bucket1 []Student // категория до 200 включительно
20     var bucket2 []Student // от 201 до 500 включительно
21     var bucket3 []Student // от 501+
22
23     for _, s := range students {
24         if s.QueueNumber <= 200 {
25             bucket1 = append(bucket1, s)
26         } else if s.QueueNumber <= 500 {
27             bucket2 = append(bucket2, s)
28         } else {
29             bucket3 = append(bucket3, s)
30         }
31     }
32
33     insertionSort(bucket1)
34     insertionSort(bucket2)
35     insertionSort(bucket3)
36
37     result := make([]Student, 0, len(students))
38     result = append(result, bucket1...)
39     result = append(result, bucket2...)
40     result = append(result, bucket3...)
41
42     return result
43 }
44
45 func insertionSort(arr []Student) {
46     for i := 1; i < len(arr); i++ {
47         key := arr[i]
48         j := i - 1
49         for j >= 0 && arr[j].QueueNumber > key.QueueNumber {
50             arr[j+1] = arr[j]
51             j--
52         }
53         arr[j+1] = key
54     }
55 }
56
```

```

57 func main() {
58     groupData := []Student{
59         {Name: "Егорова Мария", IsMagistr: true, InQueue: true, QueueNumber:
60             150},
61         {Name: "Фролова Кристина", IsMagistr: true, InQueue: false,
62             QueueNumber: 0},
63         {Name: "Костюченко Тимофей", IsMagistr: false, InQueue: true,
64             QueueNumber: 342},
65         {Name: "Портнова Ксения", IsMagistr: true, InQueue: true, QueueNumber:
66             612},
67         {Name: "Кепжаев Рахим", IsMagistr: true, InQueue: true, QueueNumber:
68             89},
69         {Name: "Даньшин Семён", IsMagistr: true, InQueue: true, QueueNumber:
70             540},
71         {Name: "Шихайло Сергей", IsMagistr: false, InQueue: false, QueueNumber:
72             0},
73         {Name: "Бабушкин Александр", IsMagistr: true, InQueue: true,
74             QueueNumber: 221},
75         {Name: "Якунин Андрей", IsMagistr: true, InQueue: true, QueueNumber:
76             431},
77         {Name: "Казанцев Александр", IsMagistr: true, InQueue: true,
78             QueueNumber: 641},
79         {Name: "Юркин Александр", IsMagistr: true, InQueue: true, QueueNumber:
80             112},
81         {Name: "Панкратова Анна", IsMagistr: false, InQueue: true, QueueNumber:
82             299},
83         {Name: "Колесникова Лариса", IsMagistr: true, InQueue: false,
84             QueueNumber: 0},
85     }
86 }
87
88 fmt.Println("Исходный список:")
89 for s := range groupData {
90     fmt.Printf("Студент: %-20s | Очередь: %d\n", s.Name, s.QueueNumber)
91 }
92 fmt.Println()
93
94 sortedData := bucketSortStudents(groupData)
95
96 fmt.Println("Отсортированный список:")
97 for s := range sortedData {
98     fmt.Printf("Студент: %-20s | Очередь: %d\n", s.Name, s.QueueNumber)
99 }
100 }

```

```

→ 04_heap_sort cd ..
→ lab-03-sorting cd 05_bucket_sort
→ 05_bucket_sort go run main.go
Исходный список:
Студент: Егорова Мария | Очередь: 150
Студент: Фролова Кристина | Очередь: 0
Студент: Костюченко Тимофей | Очередь: 342
Студент: Портнова Ксения | Очередь: 612
Студент: Кепжаев Рахим | Очередь: 89
Студент: Даньшин Семён | Очередь: 540
Студент: Шихайло Сергей | Очередь: 0
Студент: Бабушкин Александр | Очередь: 221
Студент: Якунин Андрей | Очередь: 431
Студент: Казанцев Александр | Очередь: 641
Студент: Юркин Александр | Очередь: 112
Студент: Панкратова Анна | Очередь: 299
Студент: Колесникова Лариса | Очередь: 0

```

```

Отсортированный список:
Студент: Фролова Кристина | Очередь: 0
Студент: Шихайло Сергей | Очередь: 0
Студент: Колесникова Лариса | Очередь: 0
Студент: Кепжаев Рахим | Очередь: 89
Студент: Юркин Александр | Очередь: 112
Студент: Егорова Мария | Очередь: 150
Студент: Бабушкин Александр | Очередь: 221
Студент: Панкратова Анна | Очередь: 299
Студент: Костюченко Тимофей | Очередь: 342
Студент: Якунин Андрей | Очередь: 431
Студент: Даньшин Семён | Очередь: 540
Студент: Портнова Ксения | Очередь: 612
Студент: Казанцев Александр | Очередь: 641
→ 05_bucket_sort

```

1.3 Задание 3

Условие:

Оцените достоинства, недостатки и сложность изученных методов сортировок.

Оценка:

Название алгоритма	Худшее время	Память	Вывод
Быстрая	$O(n^2)$	$O(\log n)$	Раскидывает студентов по 3 корзинам вокруг центрального pivot. Если список уже отсортирован, а мы взяли крайний элемент - сервер ляжет.
Слиянием	$O(n \log n)$	$O(n)$	Рекурсивно пилит массив строго пополам. Скорость стабильная, но жестко транжирит память под копии слайсов.
Расчёской	$O(n^2)$	$O(1)$	Моя расческа уменьшает шаг на коэффициент 1.3, быстро выталкивая мелкие номера из хвоста в начало. Меняет всё прямо на месте.
Пирамидальная	$O(n \log n)$	$O(1)$	Строит двоичное дерево, где худший номер очереди сам всплывает на вершину кучи. Работает быстро и не просит лишней памяти.
Блочная	$O(n^2)$	$O(n)$	Распределяет ребят строго по трем корзинам: до 200, до 500 и хвост. Если диапазоны очереди поплывут - бакеты станут неэффективными.

Quick Sort и Heap Sort:

Если в системе критически мало оперативной памяти (например, крутим микросервис в жестких контейнерах), то строить кучу из студентов или бить их на центральные опорные элементы - лучший выбор. Лишней памяти вообще ноль. Но у быстрой сортировки есть подлянка: если список прилетит уже отсортированным, а мы выберем опорный элемент с краю, алгоритм уйдет в $O(n^2)$. Я обошел это, выбирая элемент строго по центру слайса.

Merge Sort:

Да, она транжирит память, выделяя место под новые временные структуры. Но на реальном высоконагруженном бэкенде ИТМО, где списки очередников постоянно скачут, эта сортировка гарантирует предсказуемость. Никакой хаос в базе не заставит сервер зависнуть дольше положенного времени.

Comb Sort и Bucket Sort:

Расческа круто оптимизирует старый пузырьковый метод за счет коэффициента 1.3 и быстро раскидывает мелкие номера в начало списка, но на огромных базах она все-таки уступит слиянию. А блочная сортировка хороша только тогда, когда мы четко знаем распределение (как в моем коде: до 200, до 500 и хвост). Если диапазоны поплывут, блоки станут неэффективными, и мы свалимся в обычную сортировку вставками внутри одного огромного бакета...

ЗАКЛЮЧЕНИЕ

Эта лабораторная работа окончательно разложила по полочкам работу с массивами и структурами данных под капотом. Написание пяти совершенно разных сортировок доказало, что одну и ту же задачу упорядочивания базы данных студентов можно решить множеством путей, и у каждого будут свои инженерные компромиссы.

Я понял, что в реальной разработке нет понятия **лучший алгоритм**. Если нам важна гарантированная скорость при любых входящих данных и не жалко оперативки, то мой выбор - **сортировка слиянием**. Если ресурсы процессора и памяти ограничены до предела, то строю двоичную кучу и использую **пирамидальный метод**. Главное это уметь видеть эти развилки еще на этапе проектирования архитектуры, чтобы проект держал нагрузку, а пользователи не ждали свои ответы вечно!

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Бхаргава А. Грокаем алгоритмы. Второе издание, 2026.
2. Документация по языку программирования Go, ссылка: <https://go.dev/doc/>
3. Лекция 3: Алгоритмы сортировки, Университет ИТМО